Digital Logic Structures

CSE 100/1000/1100: Introduction to Computer Engineering

Instructor:
Fatma CORUT ERGİN

Slides adapted from

- *McGraw Hill Companies, and*
- *Walid A. Najjar & Brian J. Linard*

slides

Transistor: Building Block of Computers

Microprocessors contain billions of transistors

- Core i7 Haswell-E – 2014 (2.6 billion)
- Xeon Platinum 8180 – 2017 (8 billion)
- Apple M1 Ultra – 2022 (114 billion)

Logically, transistors act as voltage-controlled switch

Combined to implement logic functions

- **AND, OR, NOT**

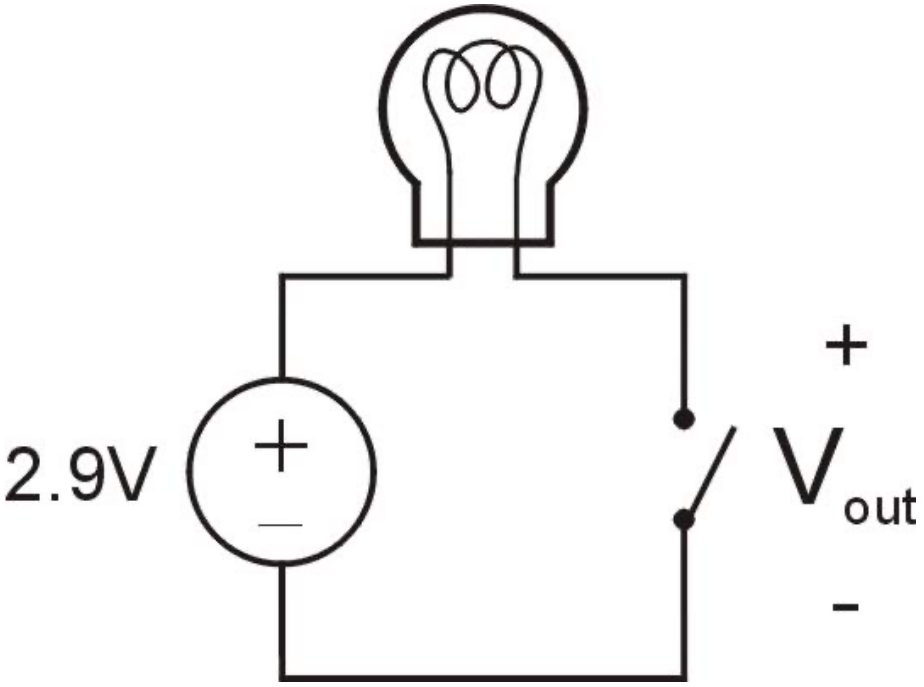
Combined to build higher-level structures

- **Adder, multiplexer, decoder, register, …**

Combined to build processor

- **LC-3**

Simple Switch Circuit



Switch open:

- **No current through circuit**
- **Light is off**
- **V_{out} is +2.9V**

Switch closed:

- **Short circuit across switch**
- **Current flows**
- **Light is on**
- **V_{out} is 0V**

Switch-based circuits can easily represent two states: on/off, open/closed, voltage/no voltage.

MOS Transistor

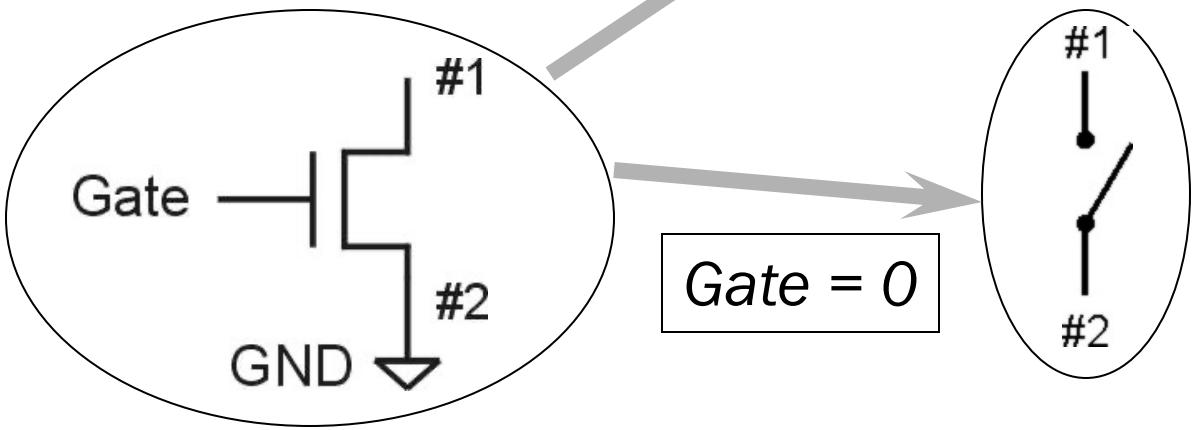
MOS = Metal Oxide Semiconductor

- two types: n-type and p-type

n-type MOS Transistor

- when Gate has positive voltage, short circuit between #1 and #2 (switch closed)
- when Gate has zero voltage, open circuit between #1 and #2 (switch open)





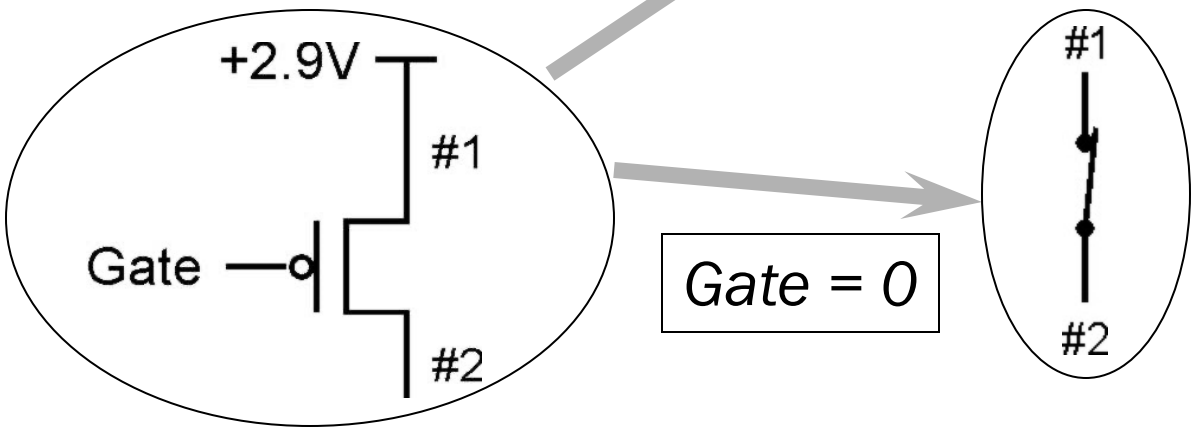
Terminal #2 must be connected to GND (0V).

p-type MOS Transistor

p-type is *complementary* to n-type

- when Gate has positive voltage, open circuit between #1 and #2 (switch open)
- when Gate has zero voltage, short circuit between #1 and #2 (switch closed)





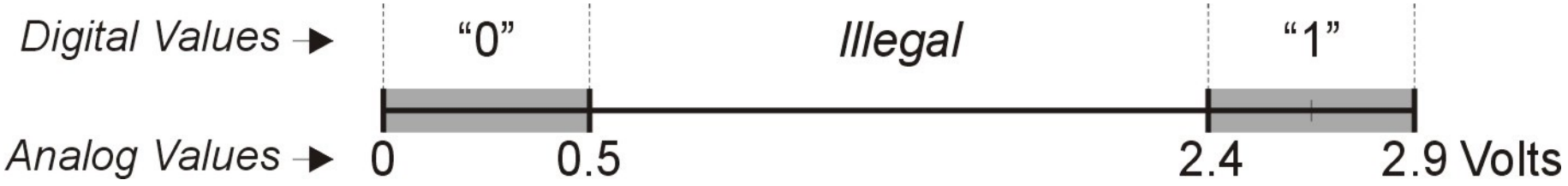
Terminal #1 must be connected to +2.9V.

Logic Gates

Use switch behavior of MOS transistors to implement logical functions: AND, OR, NOT.

Digital symbols:

- **recall that we assign a range of analog voltages to each digital (logic) symbol**



- **assignment of voltage ranges depends on electrical properties of transistors being used**
  - **typical values for "1": +5V, +3.3V, +2.9V**
  - **from now on we'll use +2.9V**

CMOS Circuit

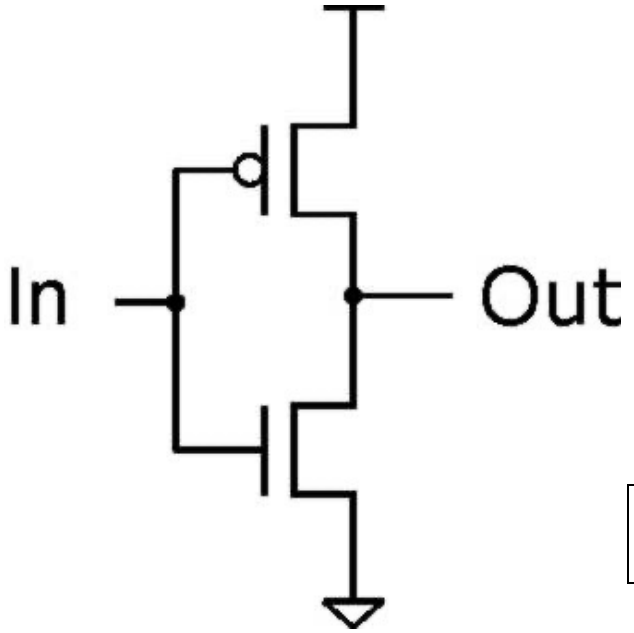
Complementary MOS

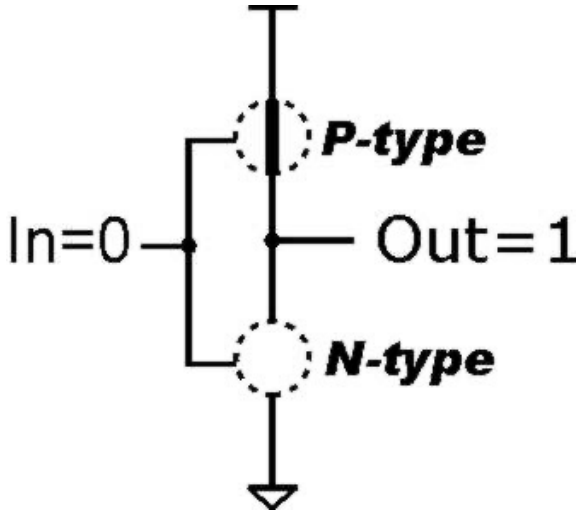
Uses both n-type and p-type MOS transistors

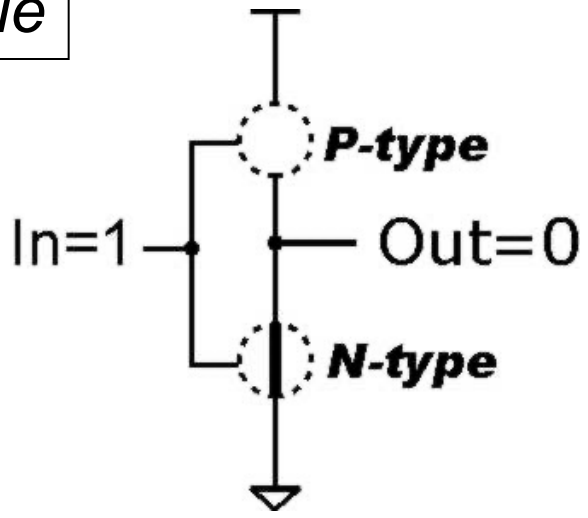
- **n-type**
  - **Attached to GND**
  - **Closed circuit when input is one**
- **p-type**
  - **Attached to + voltage**
  - **Closed circuit when input is zero**

For all inputs, make sure that output is either connected to GND or to +, but not both!

Inverter (NOT Gate)





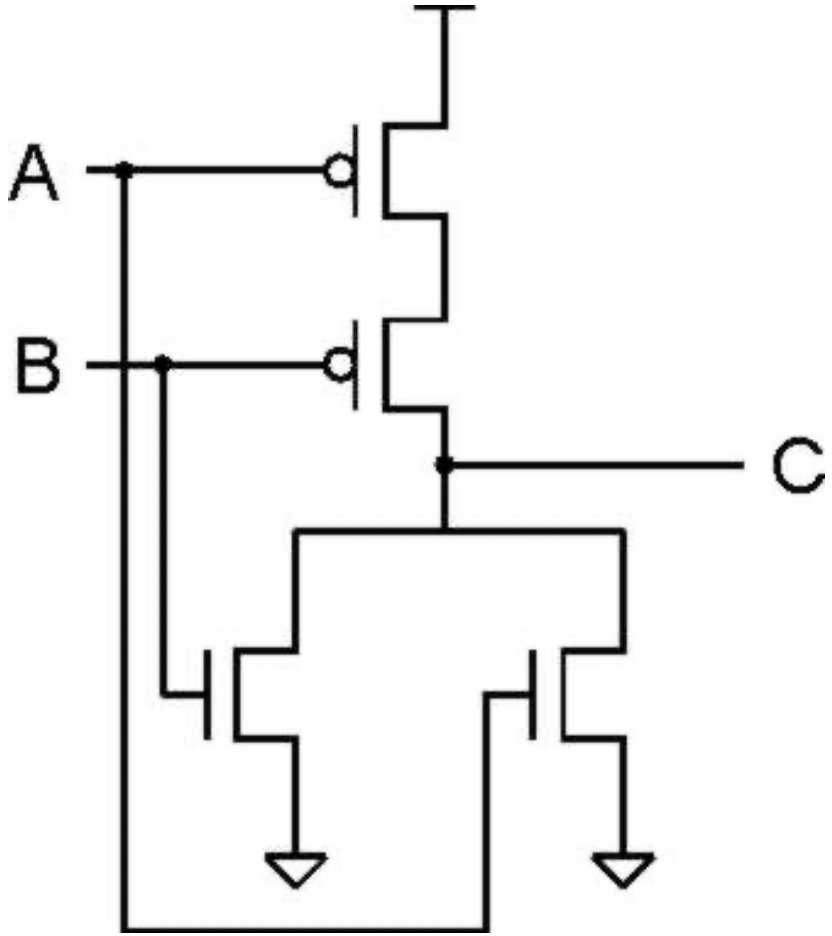


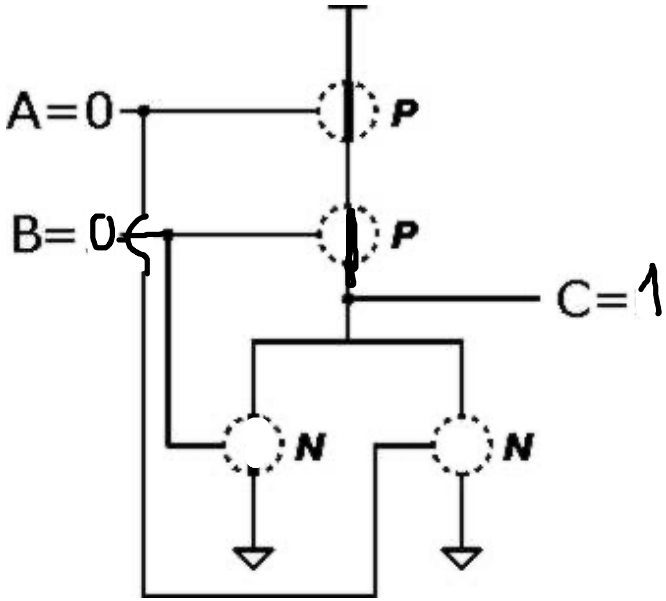
Truth table

In	Out
0 V	2.9 V
2.9 V	0 V

In	Out
0	1
1	0

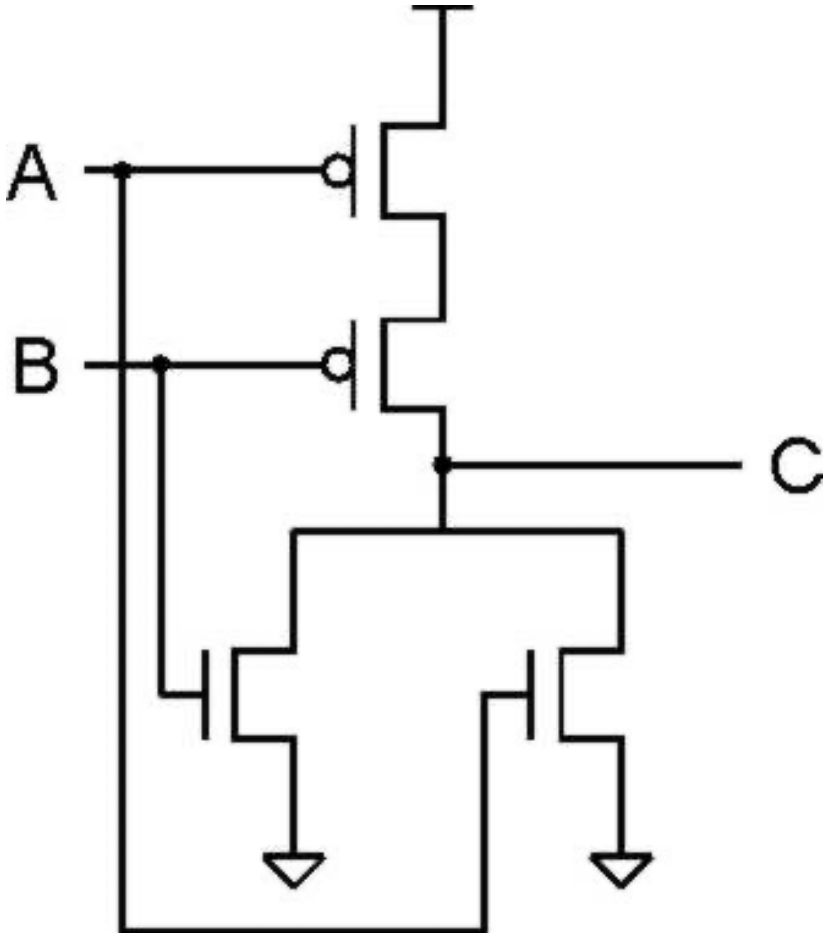
NOR Gate (NOT OR)

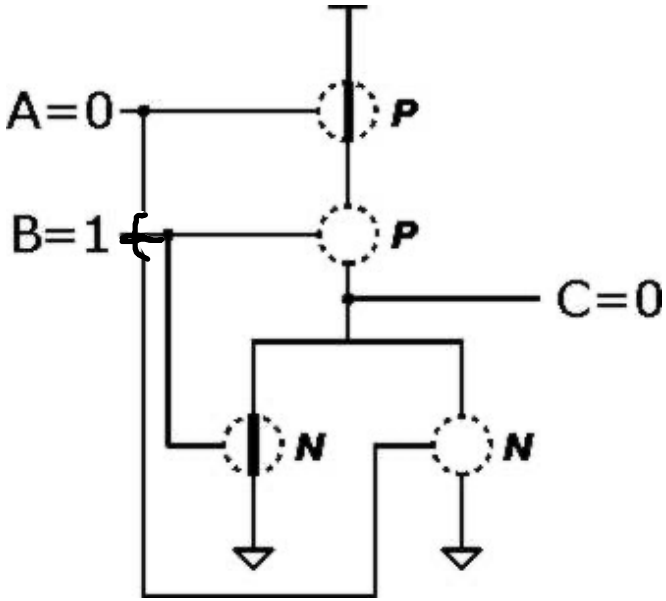




Note: Serial structure on top, parallel on bottom.

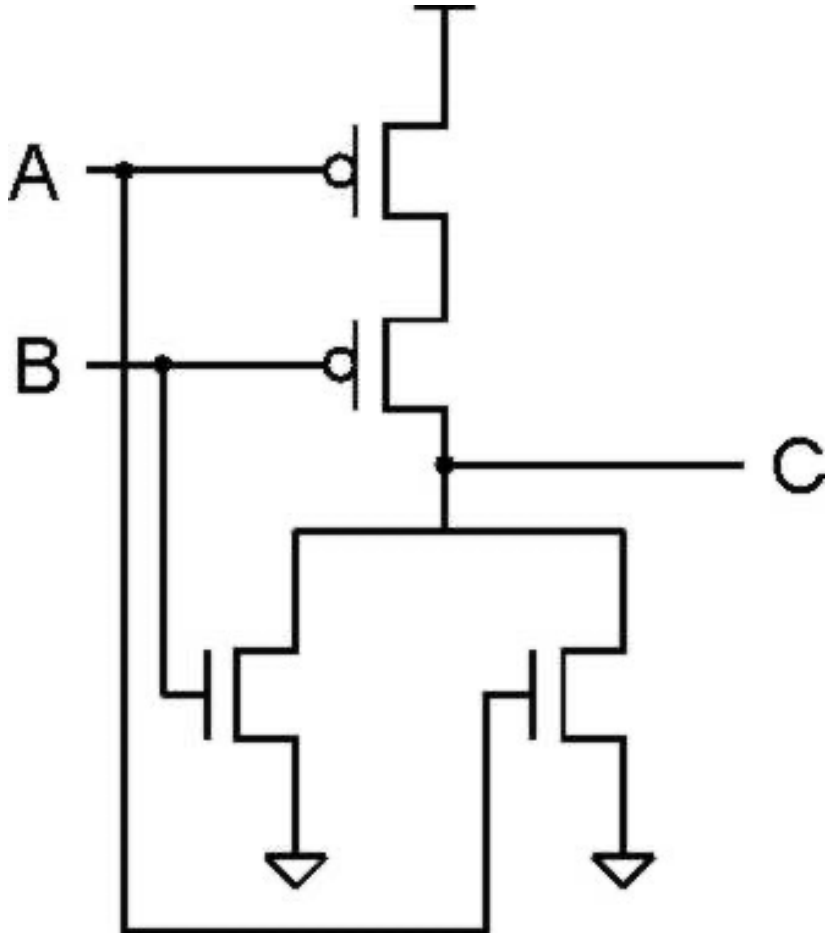
NOR Gate (NOT OR)





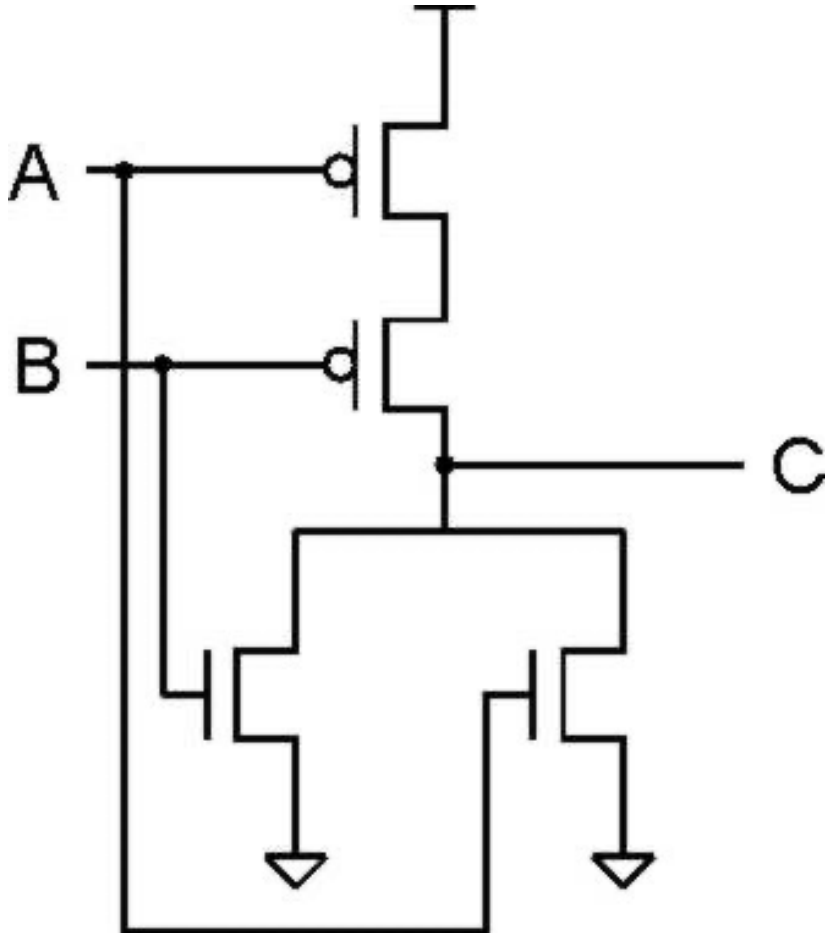
Note: Serial structure on top, parallel on bottom.

NOR Gate (NOT OR)



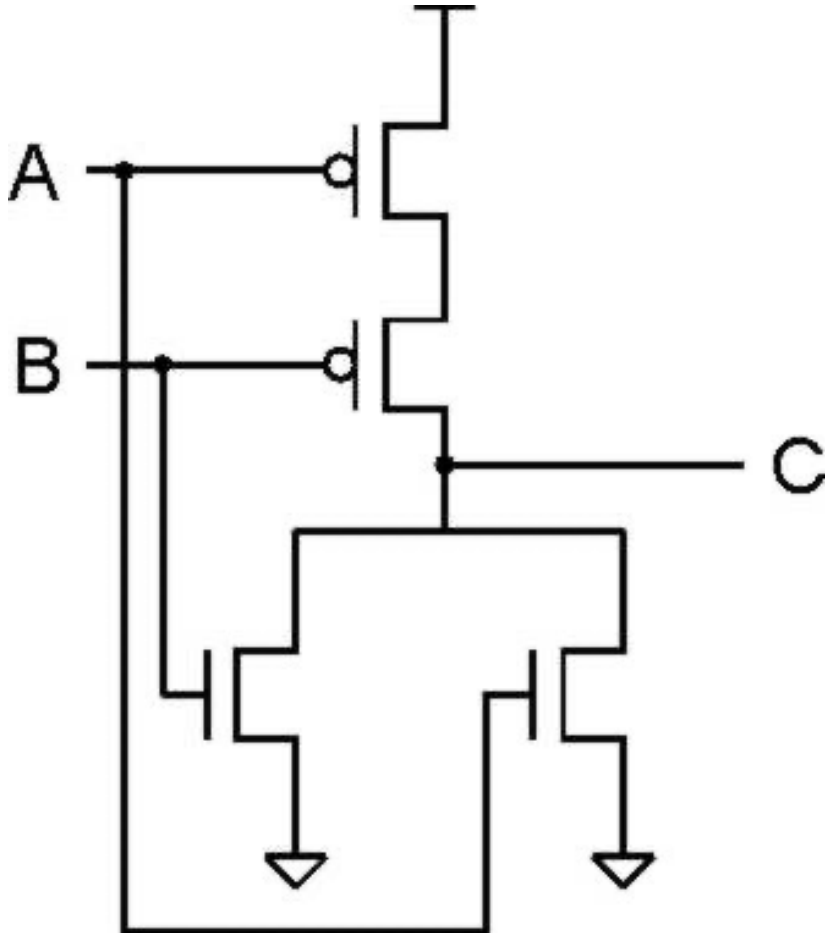
Note: Serial structure on top, parallel on bottom.

NOR Gate (NOT OR)



Note: Serial structure on top, parallel on bottom.

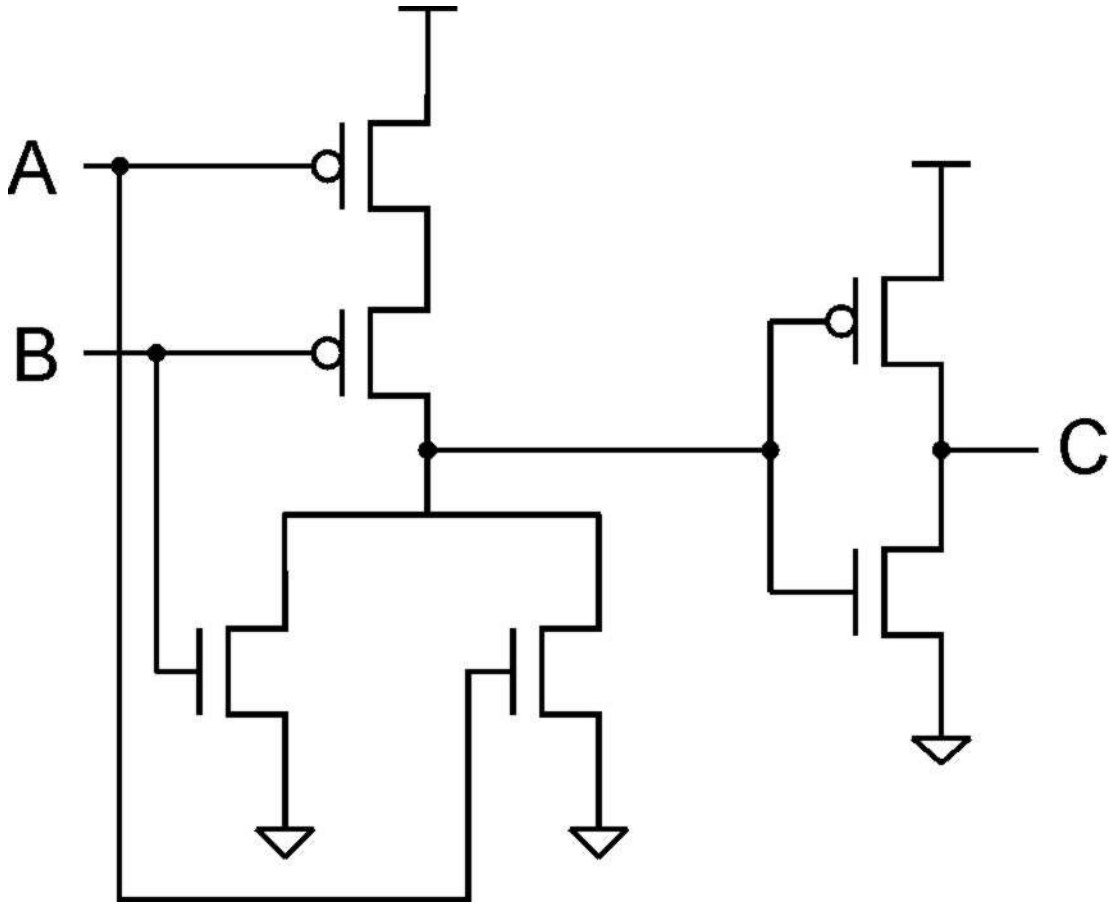
NOR Gate (NOT OR)



A	B	C
0	0	1
0	1	0
1	0	0
1	1	0

Note: Serial structure on top, parallel on bottom.

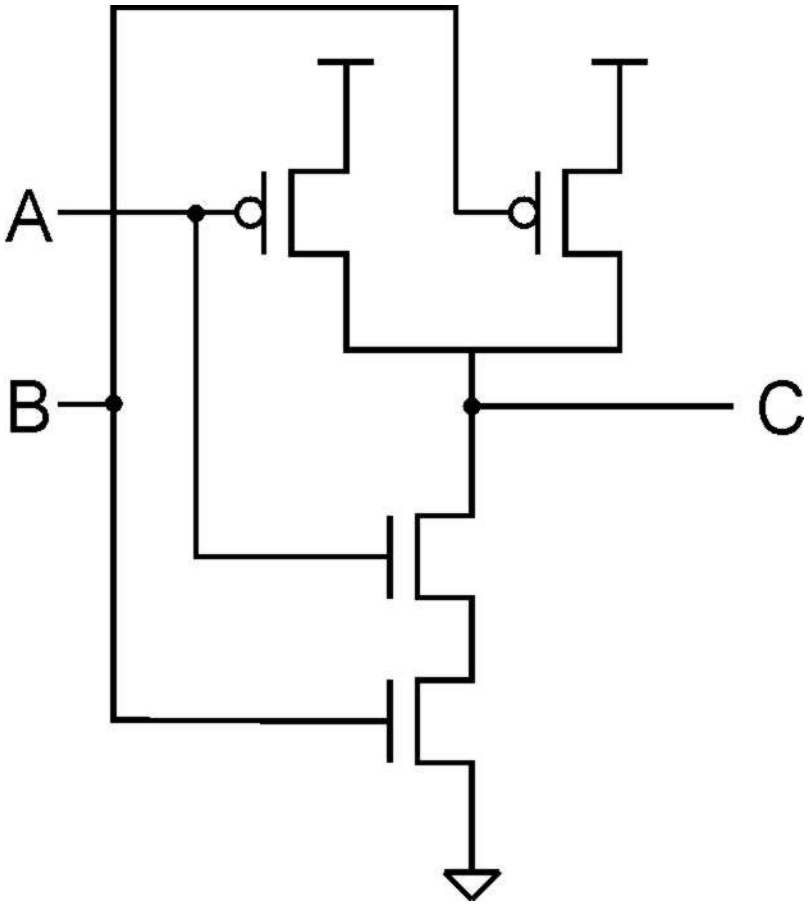
OR Gate

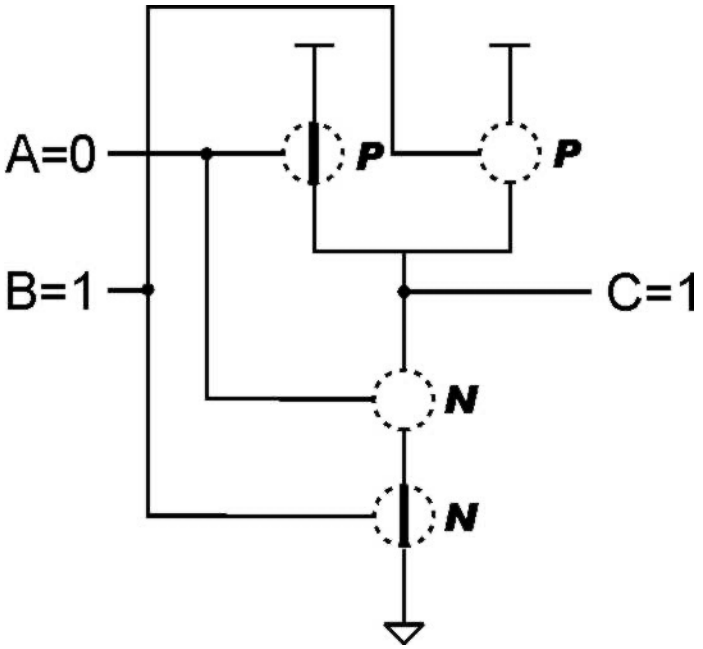


A	B	C
0	0	0
0	1	1
1	0	1
1	1	1

Add inverter to NOR.

NAND Gate (NOT AND)

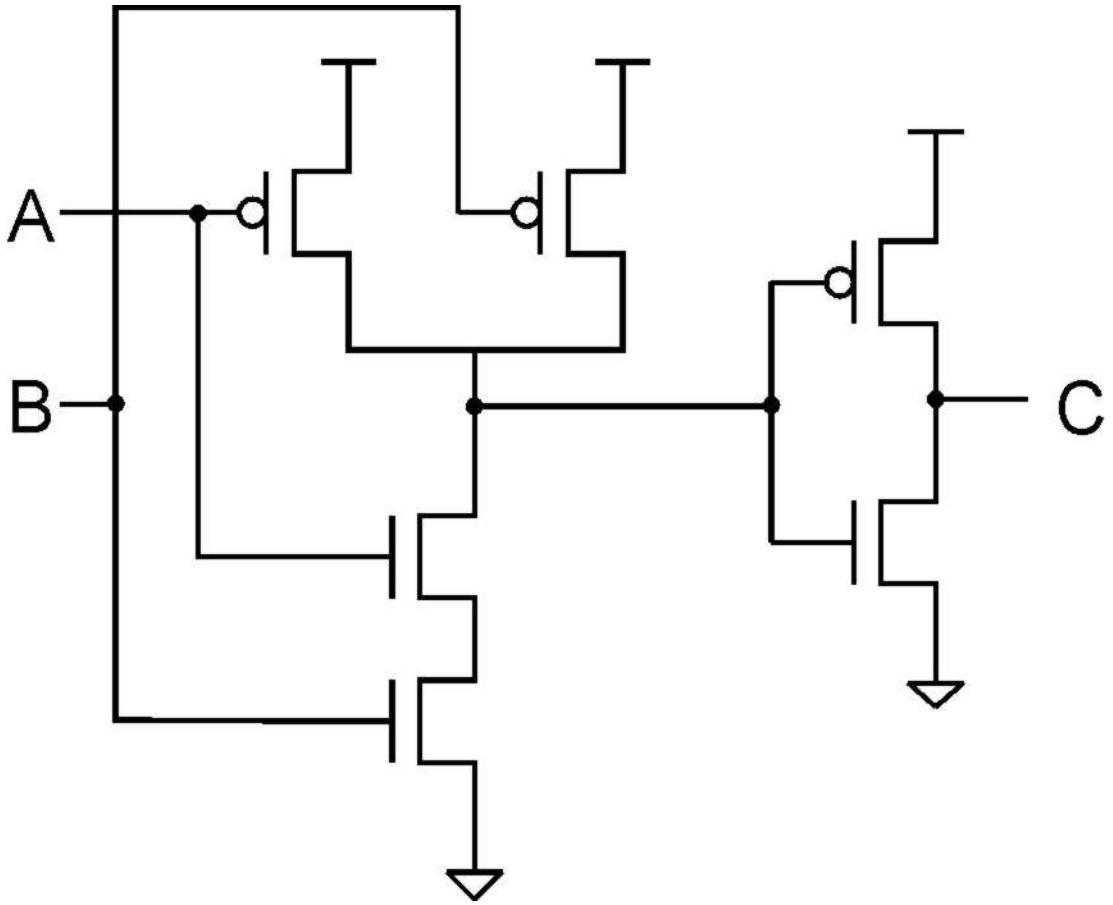




A	B	C
0	0	1
0	1	1
1	0	1
1	1	0

Note: Parallel structure on top, serial on bottom.

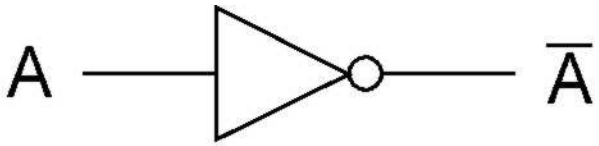
AND Gate



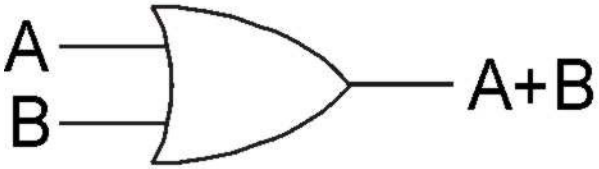
A	B	C
0	0	0
0	1	0
1	0	0
1	1	1

Add inverter to NAND.

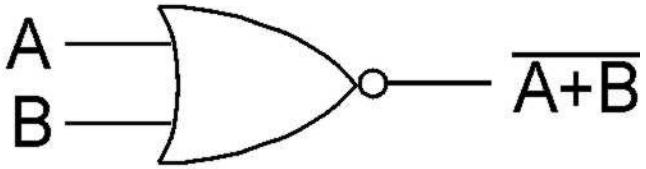
Basic Logic Gates



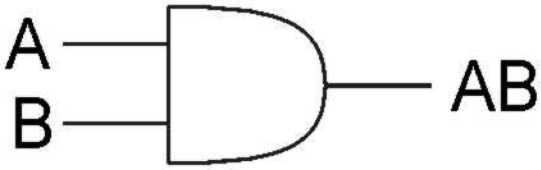
NOT



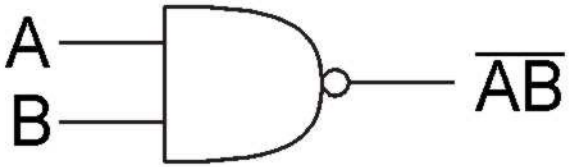
OR



NOR



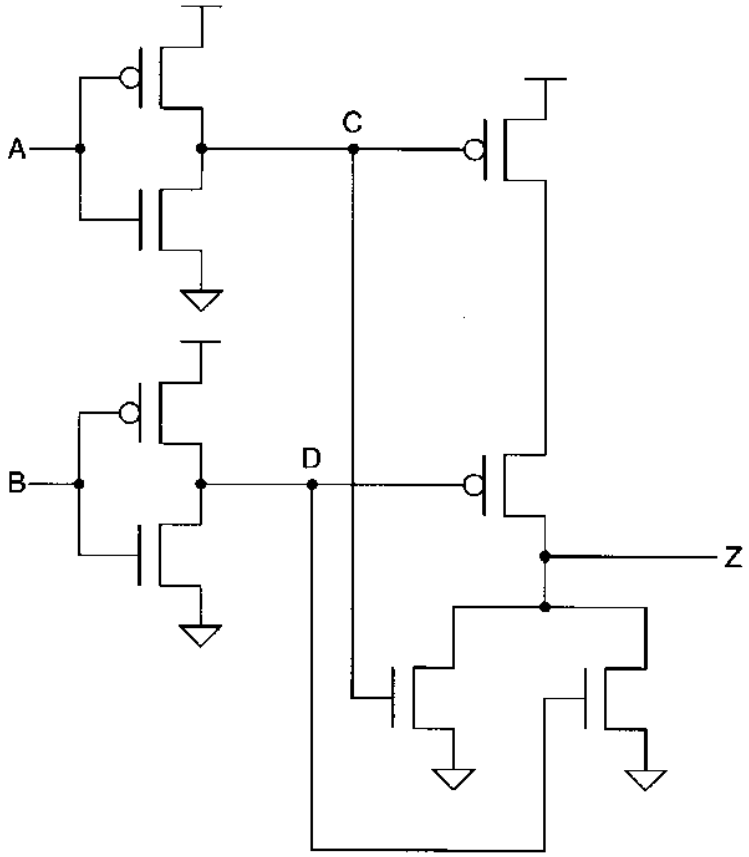
AND



NAND

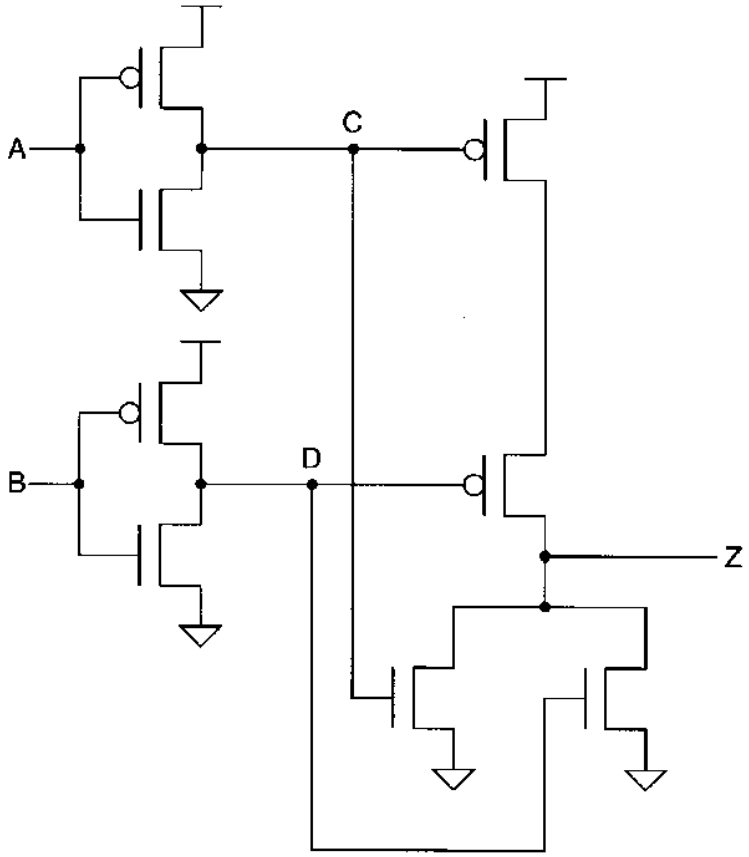
A Transistor Level Circuit Example

For the transistor-level circuit in the given figure, fill in the truth table.
What is Z in terms of A and B (write the logical equation)?



A Transistor Level Circuit Example

For the transistor-level circuit in the given figure, fill in the truth table.
What is Z in terms of A and B (write the logical equation)?



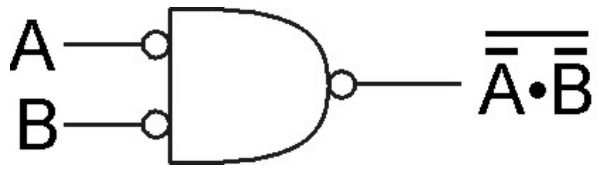
A	B	C	D	Z
0	0	1	1	0
0	1	1	0	0
1	0	0	1	0
1	1	0	0	1

Z = A AND B = (A' OR B')'

DeMorgan's Law

Converting AND to OR (with some help from NOT)

Consider the following gate:



A	B	$\overline{A}$	$\overline{B}$	$\overline{A} \cdot \overline{B}$	$\overline{\overline{A} \cdot \overline{B}}$
0	0	1	1	1	0
0	1	1	0	0	1
1	0	0	1	0	1
1	1	0	0	0	1

Same as A+B!

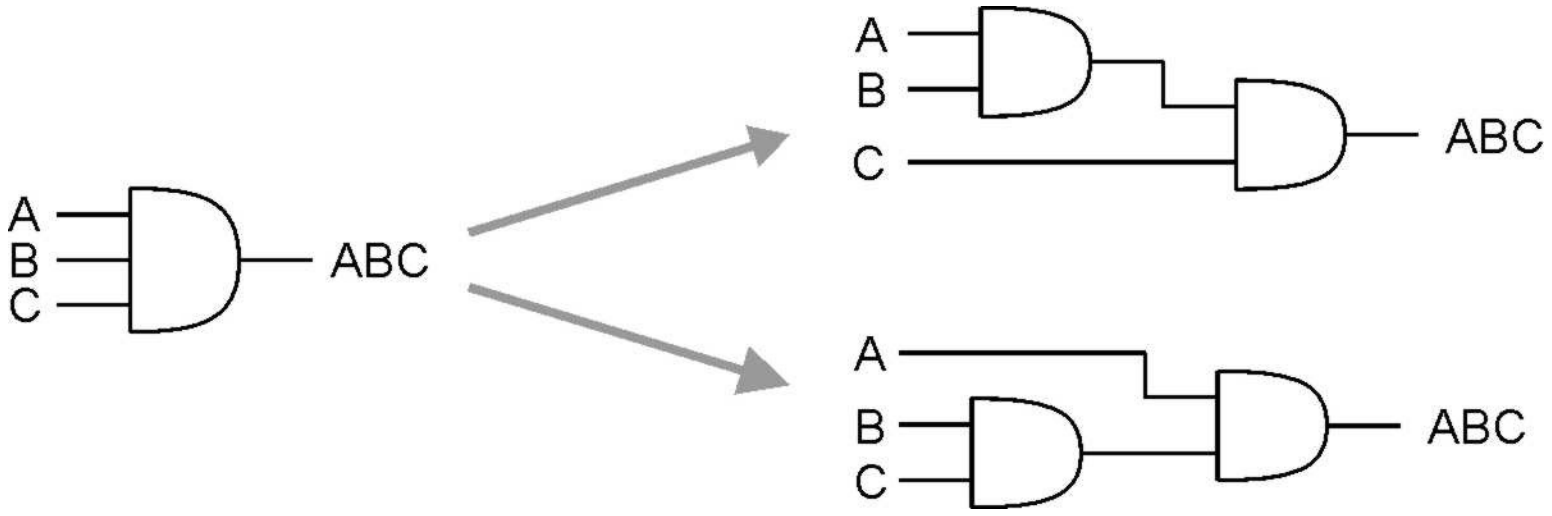
To convert AND to OR (or vice versa), invert inputs and output.

More than 2 Inputs?

AND/OR can take any number of inputs.

- **AND = 1 if all inputs are 1.**
- **OR = 1 if any input is 1.**
- **Similar for NAND/NOR.**

Can implement with multiple two-input gates, or with single CMOS circuit.



Summary

MOS transistors are used as switches to implement logic functions.

- **n-type: connect to GND, turn on (with 1) to pull down to 0**
- **p-type: connect to +2.9V, turn on (with 0) to pull up to 1**

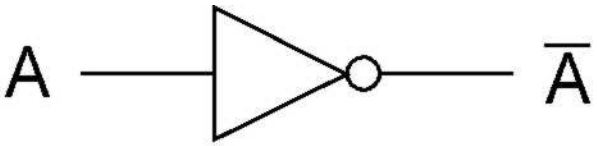
Basic gates: NOT, NOR, NAND

- **Logic functions are usually expressed with AND, OR, and NOT**

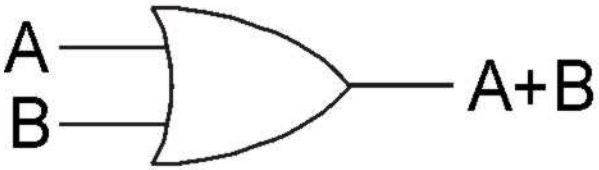
DeMorgan's Law

- **Convert AND to OR (and vice versa) by inverting inputs and output**

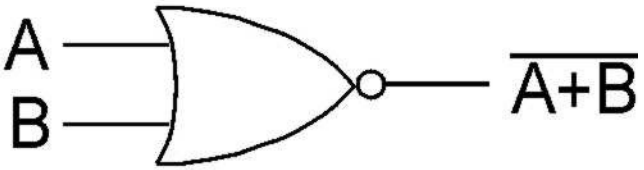
Basic Logic Gates



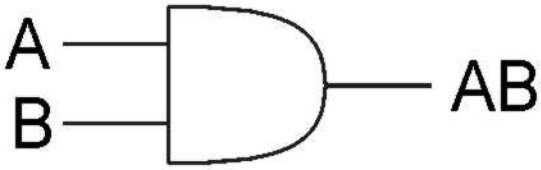
NOT



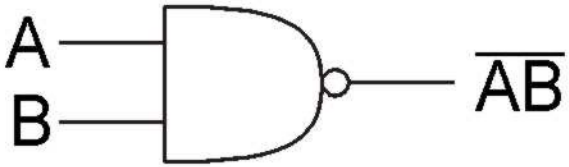
OR



NOR



AND



NAND

Building Functions from Logic Gates

Next step is to build some logic structures that are important components of the microarchitecture.

Combinational Logic Circuit

- **does not store information (stateless)**
- **output depends only on the current inputs**

Sequential Logic Circuit

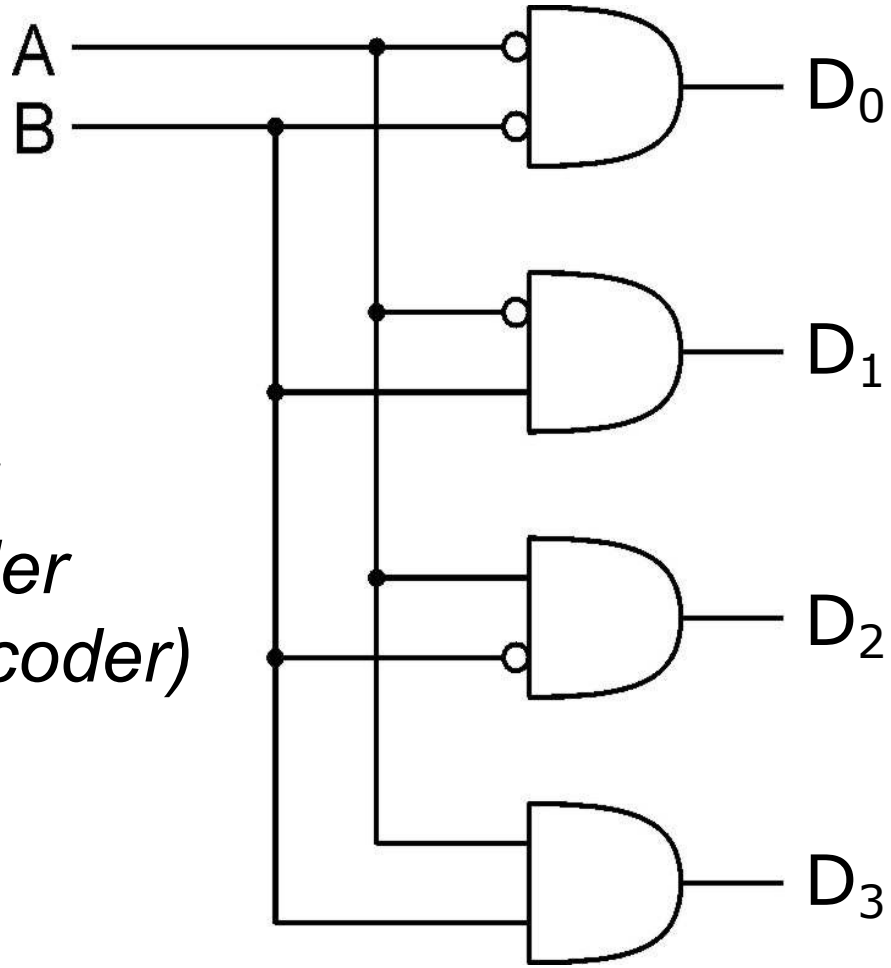
- **stores information (state) from past inputs**
- **output depends on the sequence of inputs (past and present)**

We'll look at some useful combinational circuits.

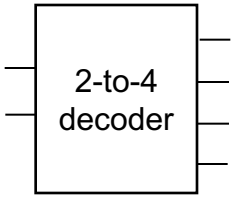
Decoder

***n* inputs, 2^n outputs**

- **exactly one output is 1 for each possible input pattern**

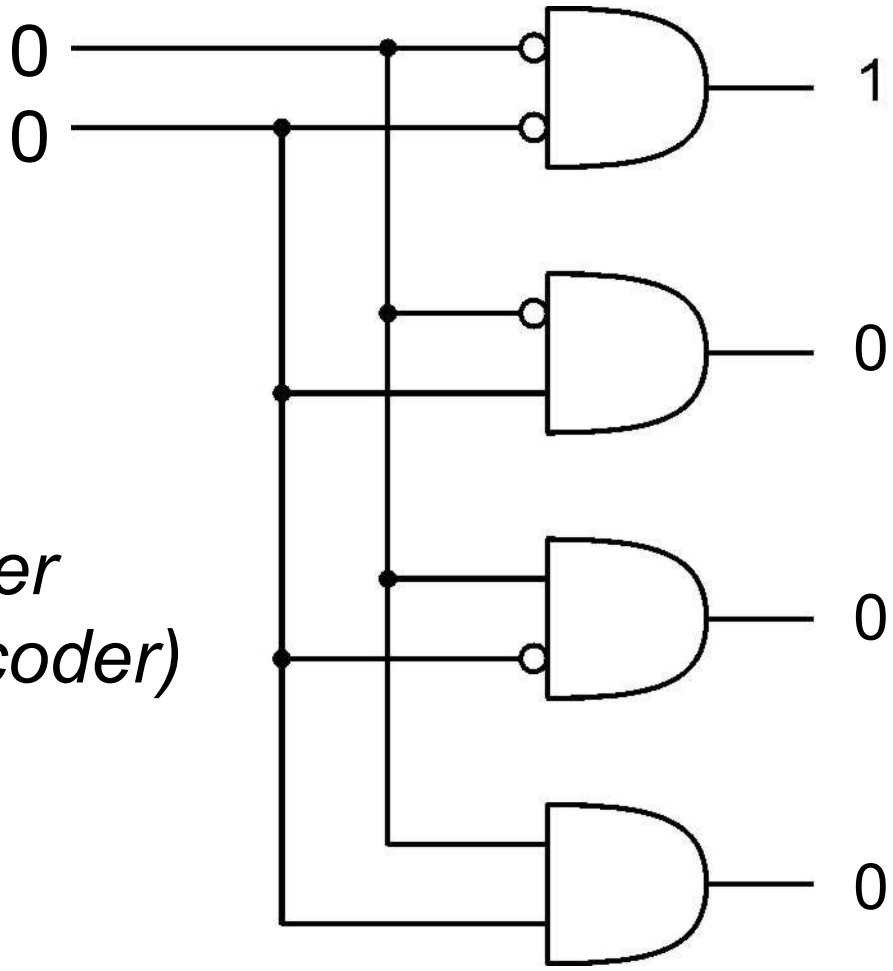


2-bit Decoder (2-to-4 decoder)



Decoder

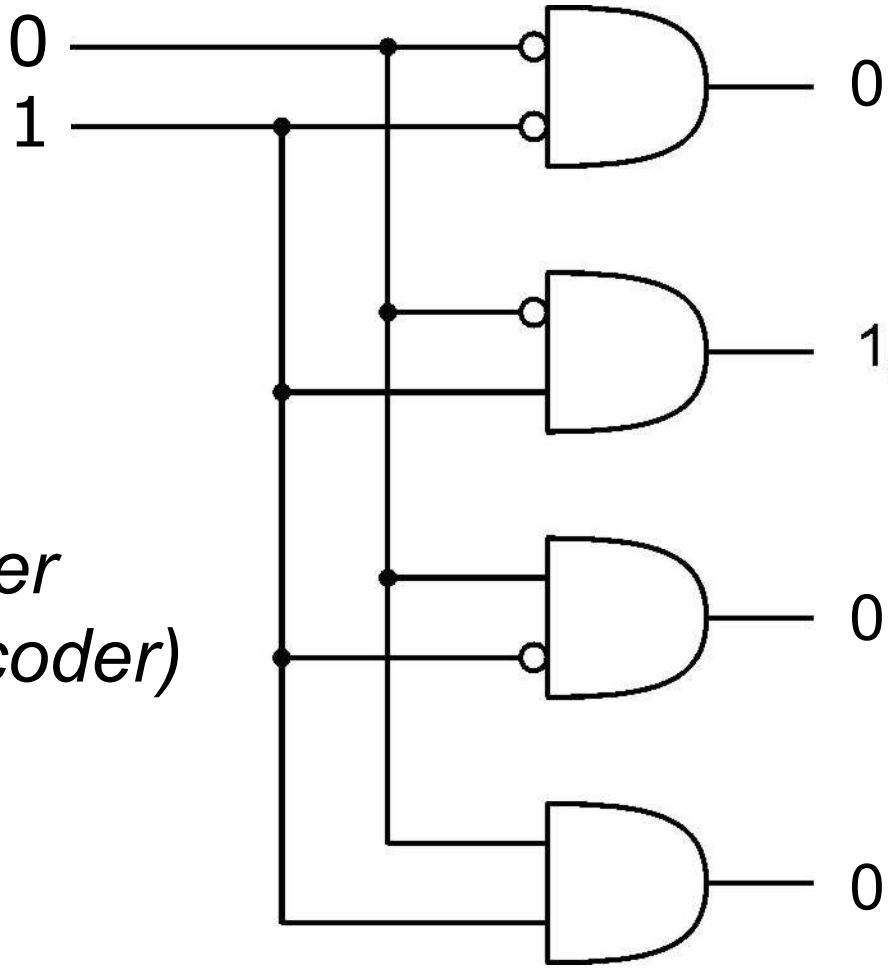
- if A = 0 and B = 0



2-bit Decoder (2-to-4 decoder)

Decoder

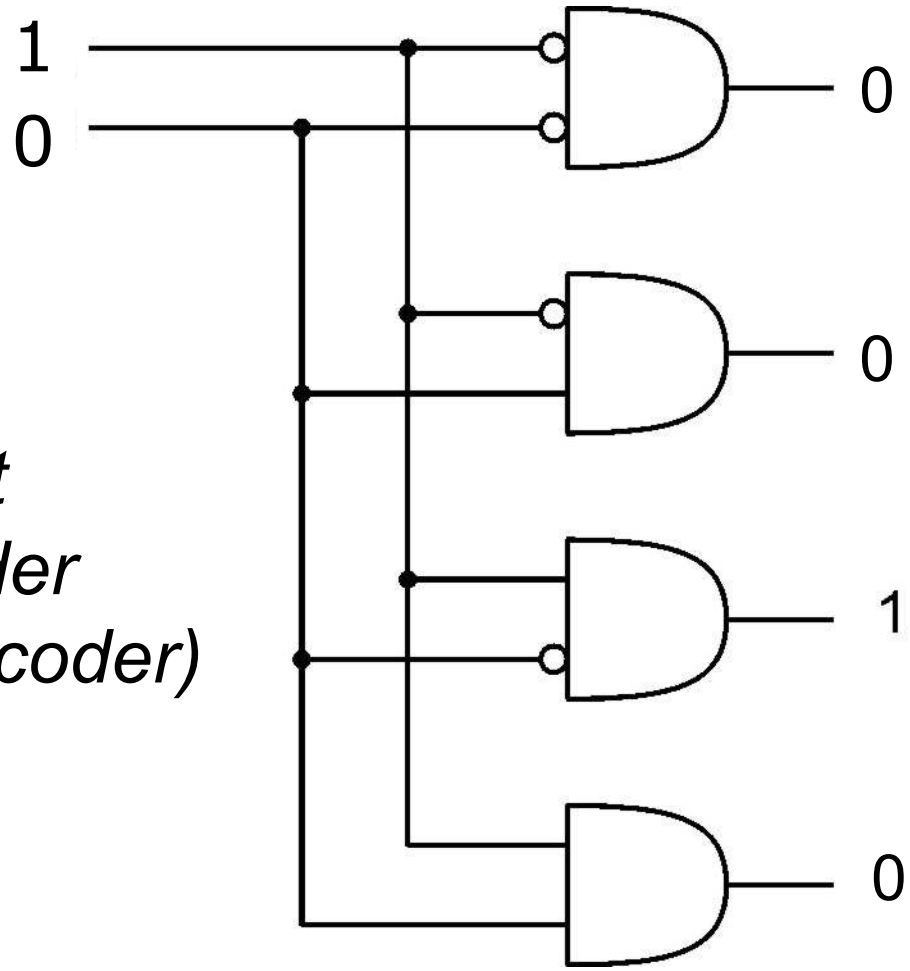
- if A = 0 and B = 1



2-bit Decoder (2-to-4 decoder)

Decoder

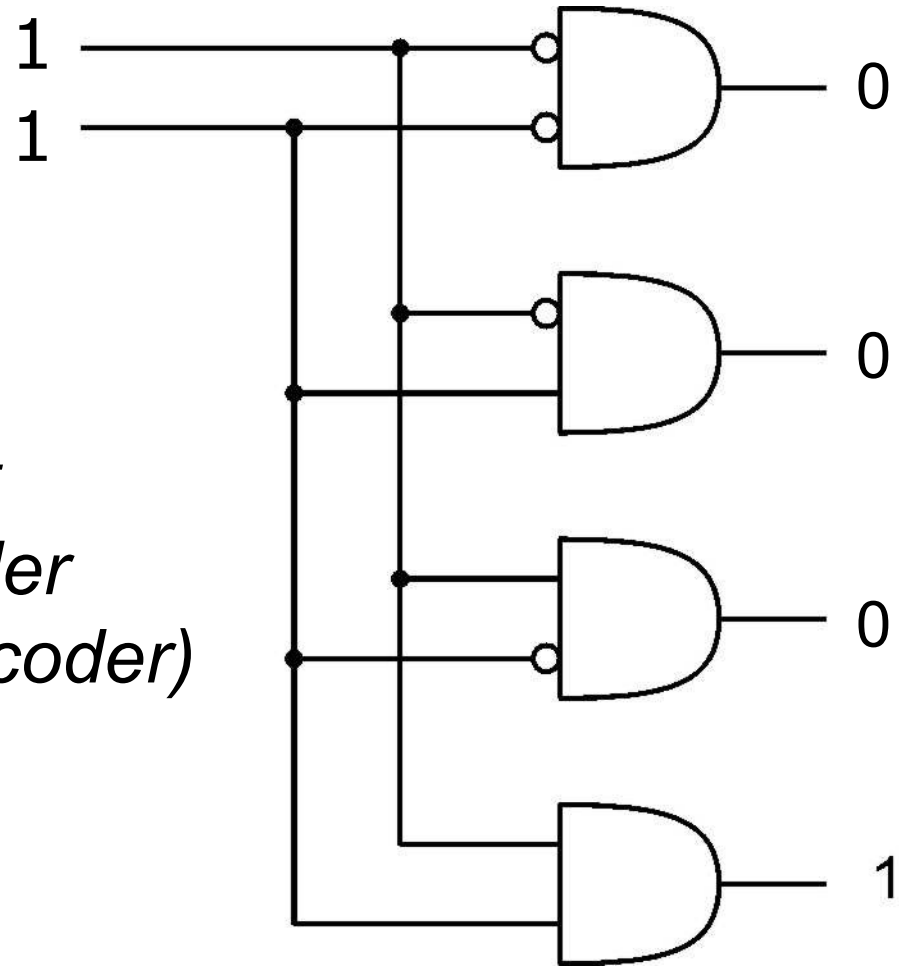
- if A = 1 and B = 0



2-bit Decoder (2-to-4 decoder)

Decoder

- if A = 1 and B = 1

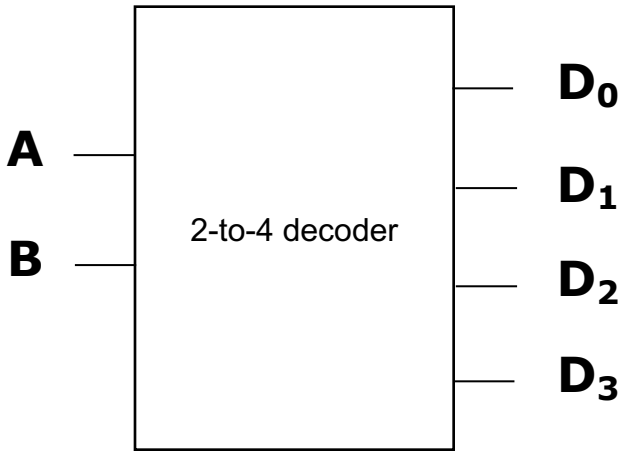


2-bit Decoder (2-to-4 decoder)

Decoder

***n* inputs, 2^n outputs**

- **exactly one output is 1 for each possible input pattern**

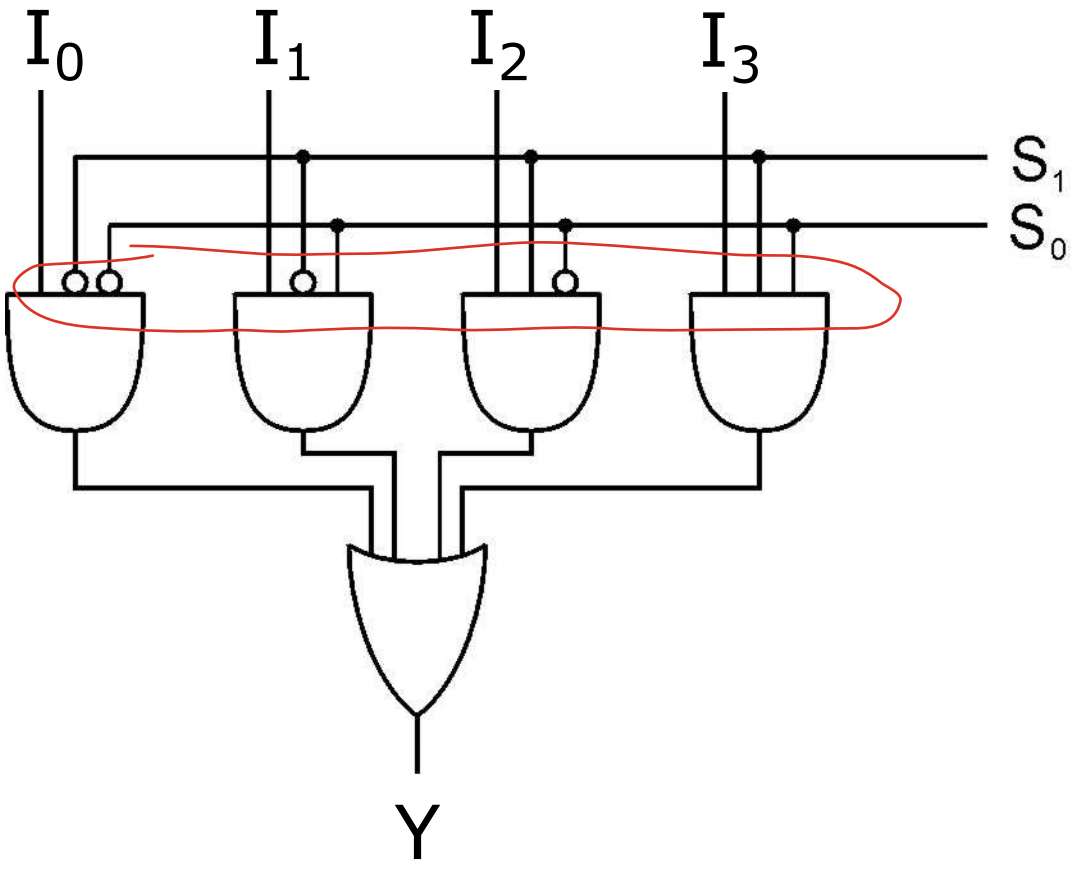


A	B	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

Multiplexer (MUX)

***n*-bit selector and 2^n inputs, one output**

- **output equals one of the inputs, depending on selector**



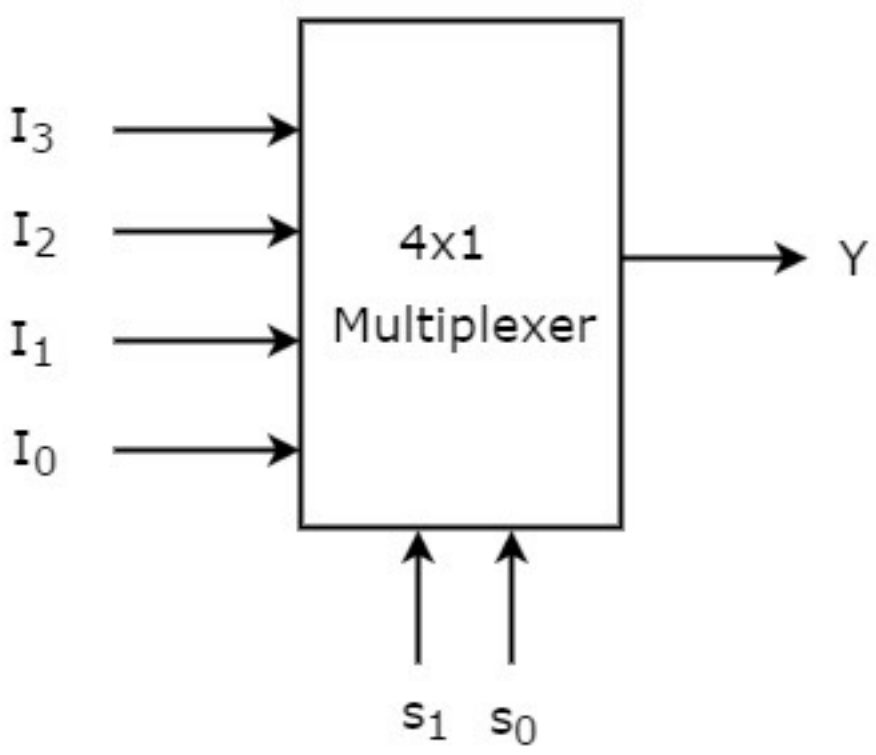
S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3

4-to-1 MUX

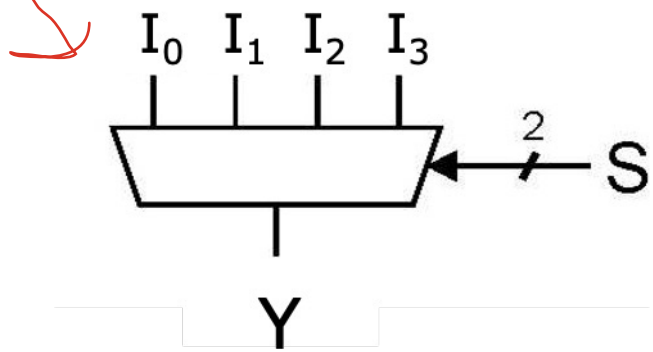
Multiplexer (MUX)

***n*-bit selector and 2^n inputs, one output**

- **output equals one of the inputs, depending on selector**



S_1	S_0	Y
0	0	I_0
0	1	I_1
1	0	I_2
1	1	I_3



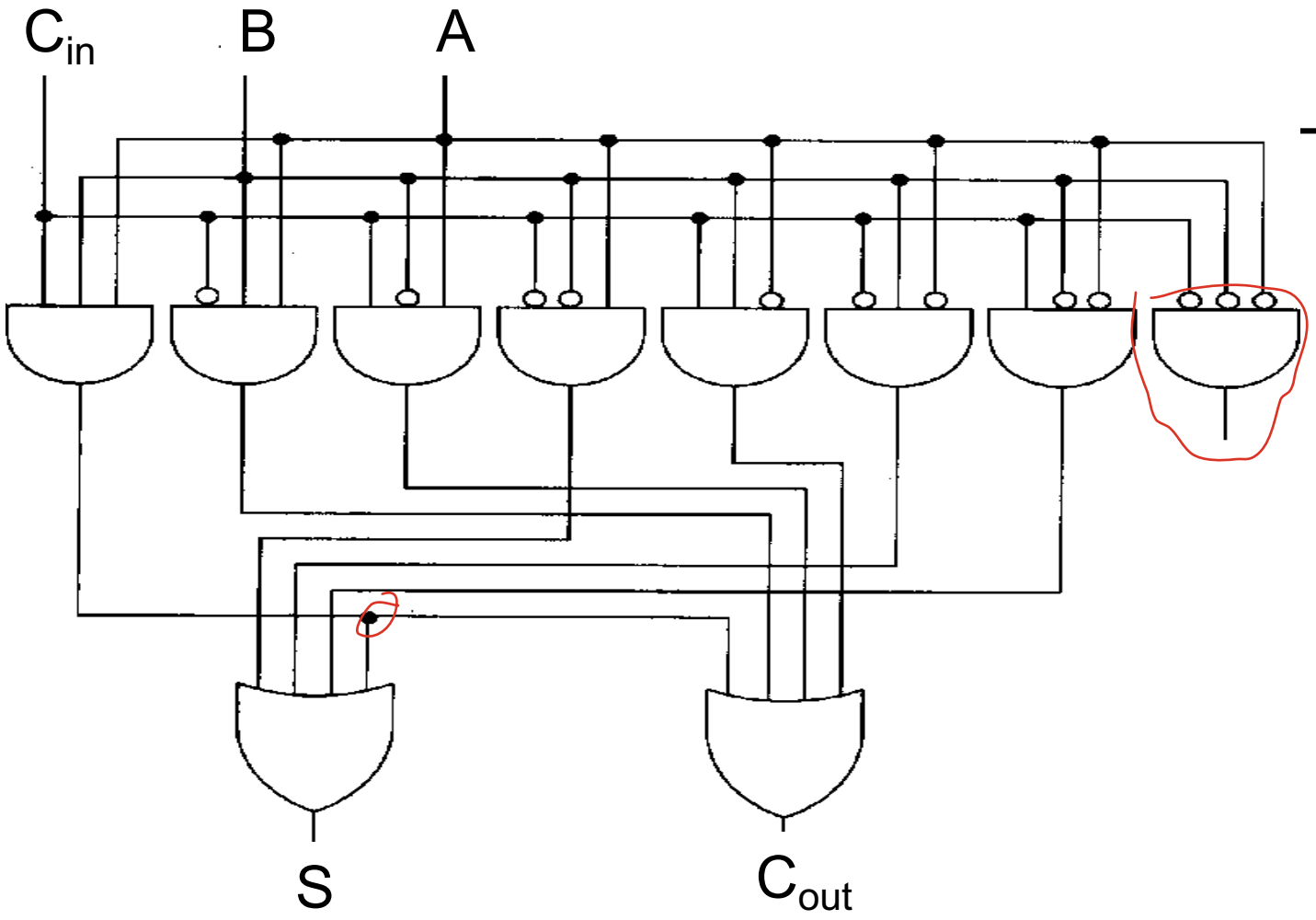
Full Adder

Add two bits (A and B) and carry-in (C_{in}), produce one-bit sum (S) and carry-out (C_{out}).

A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

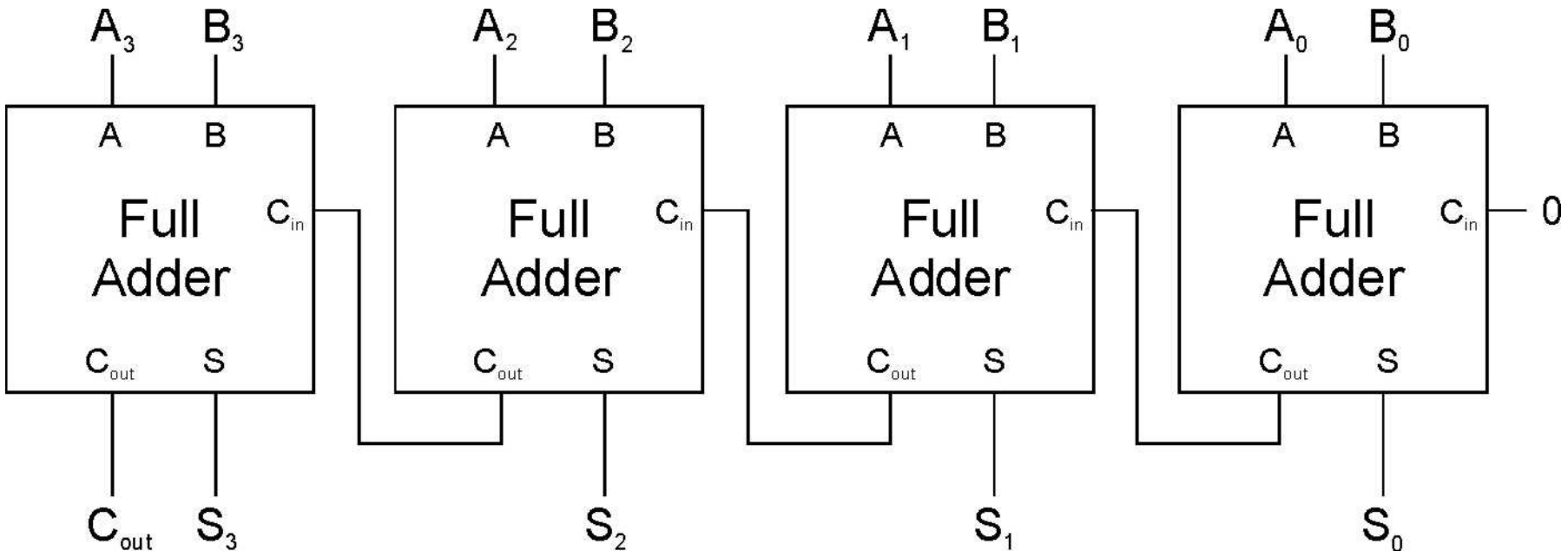
Full Adder

Add two bits (A and B) and carry-in (C_{in}), produce one-bit sum (S) and carry-out (C_{out}).



A	B	C_{in}	S	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Four-bit Adder

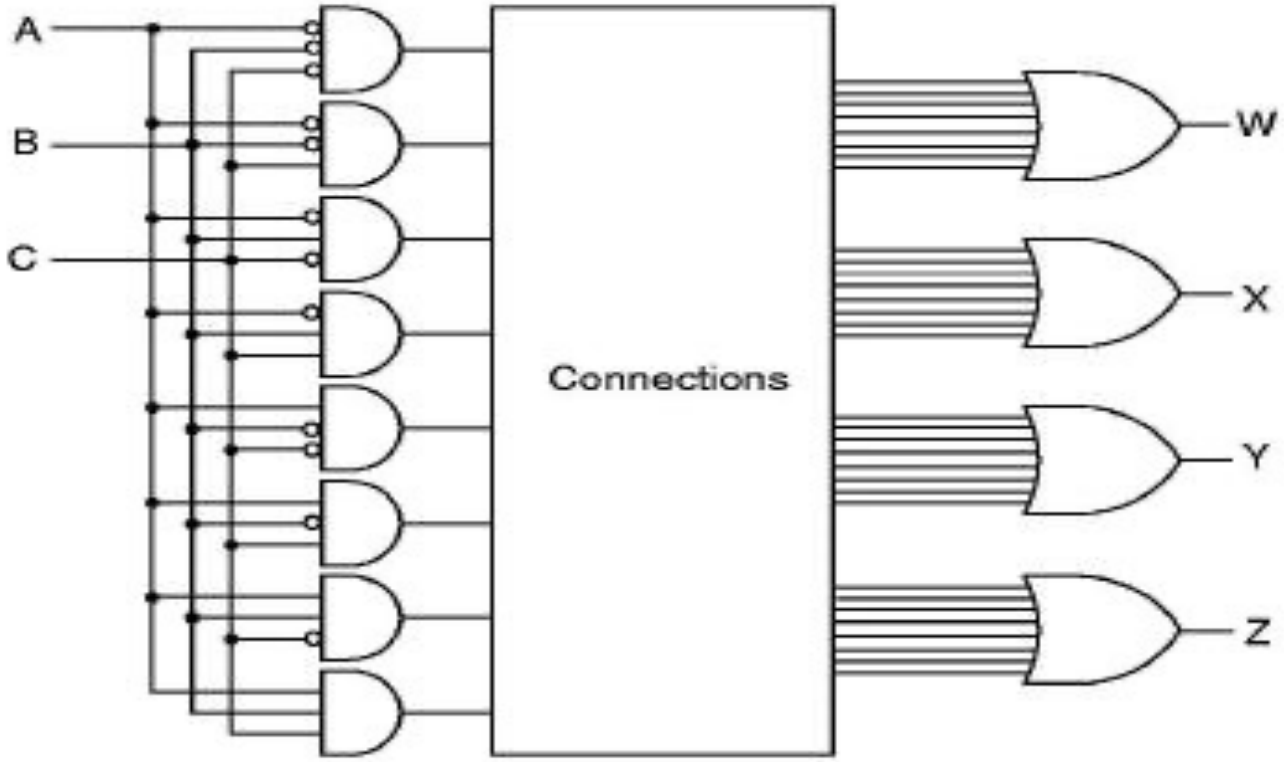


Programmable Logic Array

It is possible to build any collection of logic functions!

We program the connections

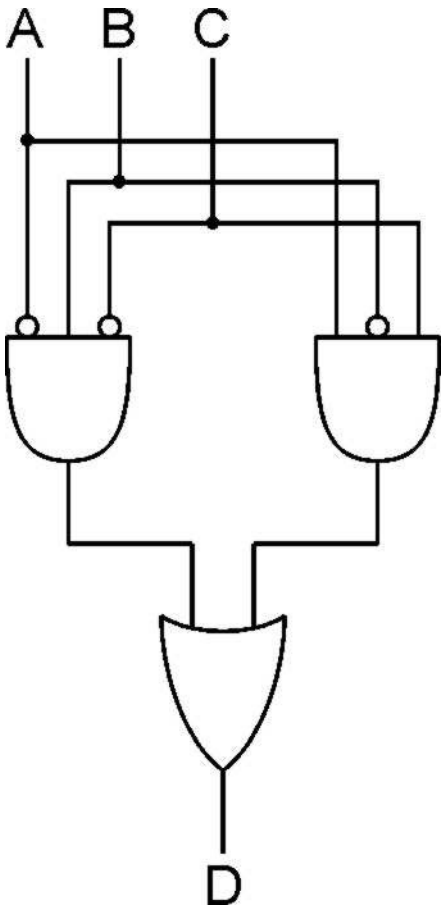
- **from AND gate outputs to OR gate inputs**



Logical Completeness

Can implement <u>ANY</u> logic function with AND, OR, NOT gates.

A	B	C	D
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	0



1. AND combinations that yield a "1" in the truth table.

2. OR the results of the AND gates.

Example

Use the truth table to answer the questions:

a) Write the logical expression which corresponds to the truth table. Do not simplify the expression.

Hint: Format your answer like:
F = (__AND__AND__) OR (__ AND __ AND __) OR …

b) Draw the logic gate-level circuit which corresponds to the truth table. Do not simplify the expression. Use AND gates, OR gates and NOT gates

A	B	C	F
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Summary

Combinational Logic Circuits:

- **Multiplexer**
- **Decoder**
- **Adder**

Combinational vs. Sequential

Combinational Circuit

- always gives the same output for a given set of inputs
  - ex: adder always generates sum and carry, regardless of previous inputs

Sequential Circuit

- stores information
- output depends on stored information (state) plus input
  - so, a given input might produce different outputs, depending on the stored information
- *example:* ticket counter
  - advances when you push the button
  - output depends on previous state
- useful for building "memory" elements and "state machines"

Memory

Using combinational logic circuits, we can build a memory

- **a logical array of stored bits.**

Address Space:

Total number of uniquely identifiable locations.

n bits allow the addressing of 2^n memory locations.

- **Example: 24 bits can address 2^{24} = 16,777,216 (i.e. 16M) locations**
- **If each location holds 1 byte (= 8 bits) then the memory is 16MB.**
- **If each location holds 4 bytes (= 32 bits) then it is 64 MB.**

Memory

Addressability

The number of bits stored in each memory location.

- **Computers are either byte**
  - **each memory location holds 8 bits (1 byte),**
- **or word addressable**
  - **each memory location holds full standard word for that computer**
  - **1 word is 32 bits in a 32-bit system, and 64 bits in a 64-bit system**

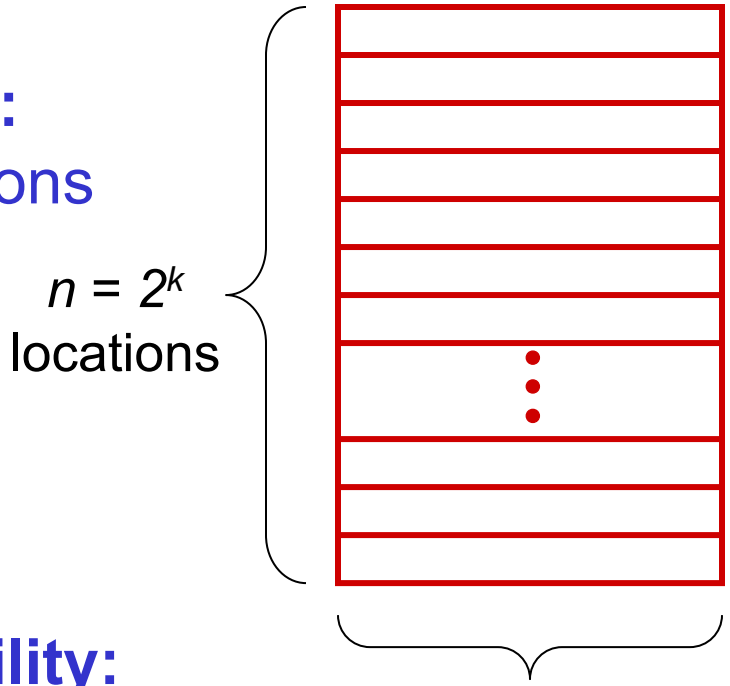
Normally, a whole word is written and read at a time:

- **If the computer is word addressable, this is simply a single address location.**

Memory

a memory – a logical array of stored bits ($2^k \times m$).

Address Space:
number of locations
(usually a power of 2)

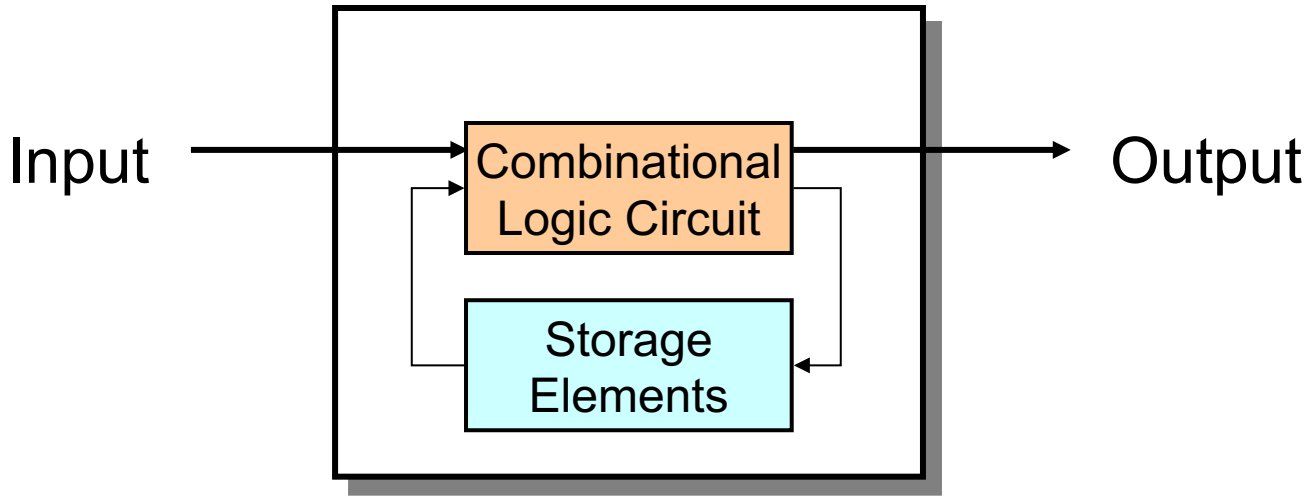


Addressability:
number of bits per location
(e.g., byte-addressable)

Example

What is the size of a memory for which 6 bits is needed to access each memory location and addressability is 3 bytes?

State Machine



A type of sequential circuit

- **Combines combinational logic with storage**
- **"Remembers" state, and changes output (and state) based on inputs and current state**
- **The output is a function of the current input and the previous state**

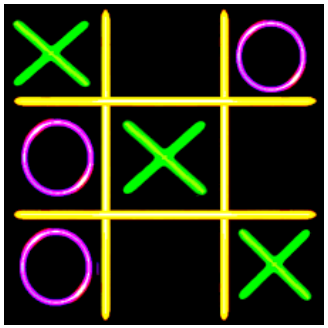
State

The state of a system is a snapshot of all the relevant elements of the system at the moment the snapshot is taken.

Examples:

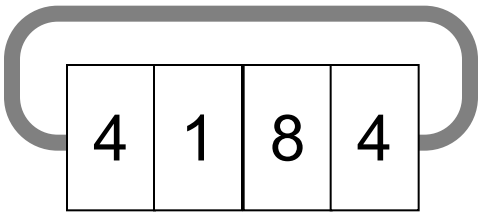
- **The state of a basketball game can be represented by the scoreboard.**
  - **Number of points, time remaining, possession, etc.**
- **The state of a tic-tac-toe game can be represented by the placement of X's and O's on the board.**

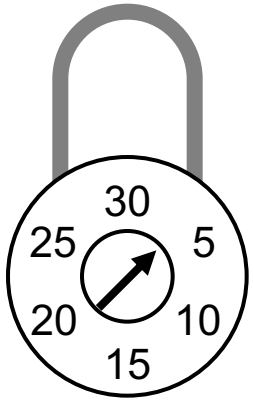




An Example: The Lock

Two types of locks





Combinational
Success depends only on the values, not the order in which they are set.

Sequential
Success depends on the sequence of values (e.g, R-13, L-22, R-3).

States of Sequential Lock

Our lock example has four different states, labelled A-D:

A: The lock is not open, and no relevant operations have been performed.

B: The lock is not open, and the user has completed the R-13 operation.

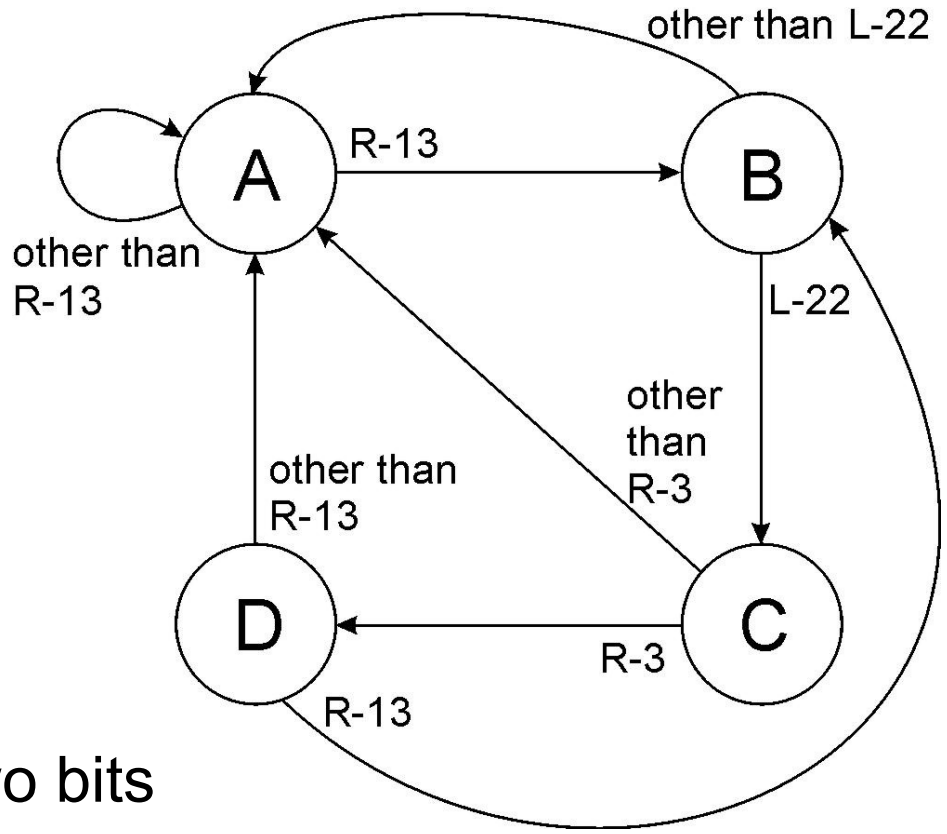
C: The lock is not open, and the user has completed R-13, followed by L-22.

D: The lock is open.

State Diagram

Shows

- **states, and**
- **actions that cause a transition between states.**



Four states – two bits

Finite State Machines

A description of a system with the following components:

1. **A finite number of states**
2. **A finite number of external inputs**
3. **A finite number of external outputs**
4. **An explicit specification of all state transitions**
5. **An explicit specification of what determines each external output value**

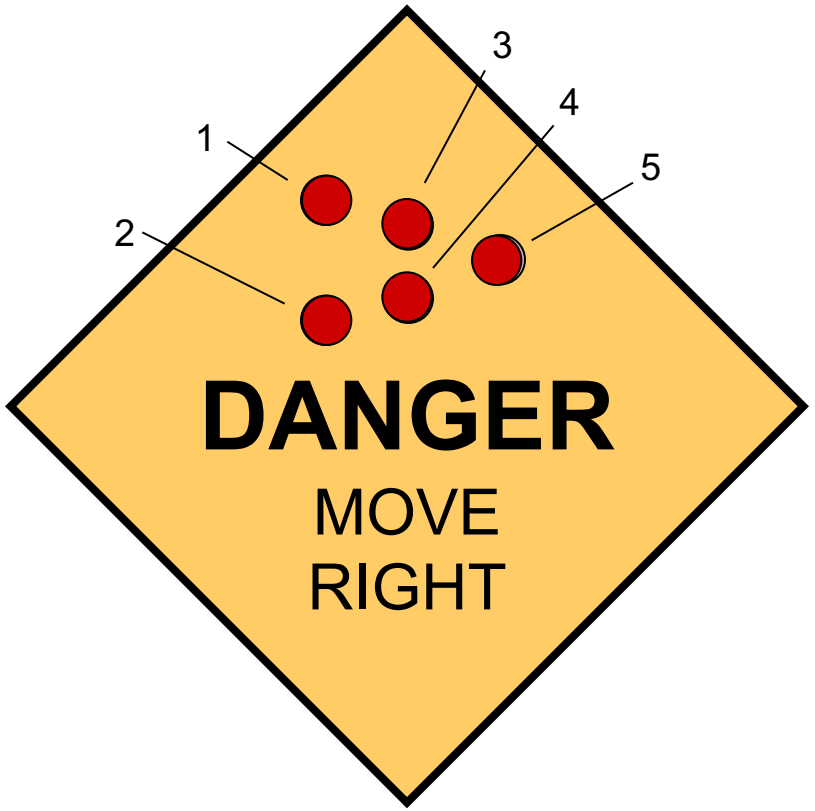
Often described by a state diagram.

- **Inputs trigger state transitions.**
- **Outputs are associated with each state (or with each transition).**

Complete Example

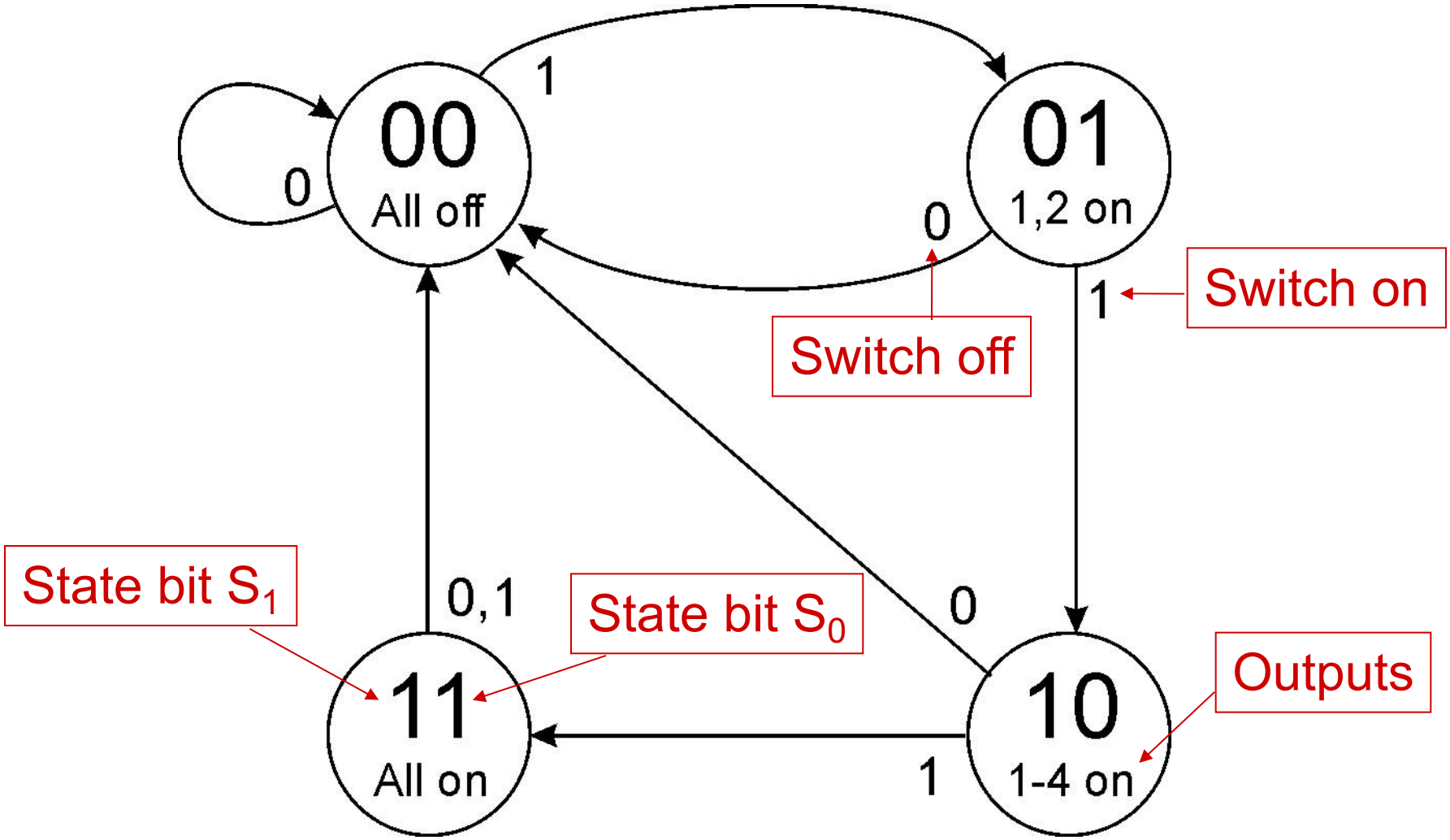
A blinking traffic sign

- **No lights on**
- **1 & 2 on**
- **1, 2, 3, & 4 on**
- **1, 2, 3, 4, & 5 on**
- **(repeat as long as switch is turned on)**



Four states – two bits

Traffic Sign State Diagram



Transition on each clock cycle.

Traffic Sign Truth Tables

Outputs
(depend only on state: S_1S_0)

Lights 1 and 2
Lights 3 and 4
Light 5

S_1	S_0	Z	Y	X
0	0	0	0	0
0	1	1	0	0
1	0	1	1	0
1	1	1	1	1

Next State: $S_1'S_0'$
(depend on state and input)

Switch

In	S_1	S_0	S_1'	S_0'
0	X	X	0	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	1
1	1	1	0	0

Whenever In=0, next state is 00.

LC-3 Data Path

Combinational Logic

Storage

State Machine

